

제18회 임베디드SW경진대회 개발완료보고서

[자율주행 모형자동차]

□ 개발 개요

○ 요약 설명

팀 명	CTRLCV
목 표	모형 자동차에 임베디드SW를 탑재하여 2차원 카메라 및 PSD 센서를 이용하여 주변 지형 및 사물을 인식하고, 스스로 정해진 차로를 주행하며 주어진 미션을 수행하는 작품을 만드는 것
소스코드	https://github.com/shythm/eswcontest-car
시연동영상	https://youtu.be/NovhfXJ-QwE

○ 개발 목적

1. 지능형운전자보조시스템(ADAS) SoC(System on Chip)를 탑재한 모형 자동차를 개발해 봄으로써 차량용 프로세서에 대해 익숙해지고 실제 현장에 적용할 수 있는 능력을 기른다.
2. 2차원 카메라를 이용한 OpenCV 라이브러리 활용 능력과 영상 처리 능력을 기른다.
3. 리눅스 기반 임베디드 시스템에서 차량 제어 알고리즘 개발을 통한 자율 주행차의 개발과 임베디드 시스템에 대한 전반적 지식을 향상한다.
4. 다수가 함께 협업해야 하는 프로젝트에서 분업을 위해 철저히 모듈화하는 설계 방법을 익힌다. 이를 통해 팀원 간의 분업 및 협업이 활발히 일어날 수 있도록 하며, 충돌 가능성을 줄여 안정적인 소프트웨어를 구현한다.
5. 주변 상황을 수학적으로 모델링하고 모형 자동차에 적용함으로써 좀 더 안정적인 주행을 구현하고 수학적 사고력을 기른다.
6. 주행 중 나타날 수 있는 각종 장애물과 다양한 도로 형태에 유연하게 대처하는 로봇을 만들어봄으로써 실제 현장에서 어떠한 어려움이 있는지 직접 체험해 본다.

○ 개발 목표

1. 인식(Recognize)과 처리(Process) 및 제어(Control) 과정을 모듈화한다.
2. 2차원 영상 처리는 적절한 수학적 필터를 사용하여 외부 환경에 빠르게 반응하고 매끄럽게 주행하는 로봇을 만든다.
3. 차선을 인식할 때 Perspective Transform, Rol, Binarization 등의 알고리즘을 이용하여 주변 상황의 영향을 최소화 시키고 정확히 차선을 인식할 수 있도록 한다.
4. 미션을 인식과 수행 상태로 나누어 이를 유한 상태 기계로 구현한다.
5. 멀티 프로세스 기법을 활용하기 위해 자원을 효율적으로 관리할 수 있도록 한다.

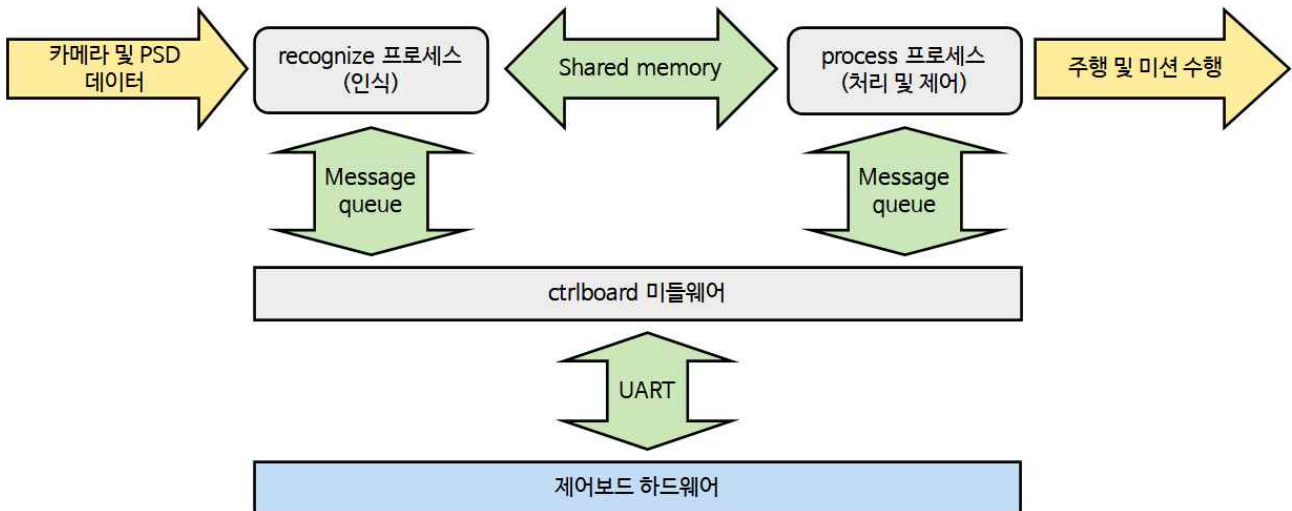
□ 개발 방향 및 전략

○ 기술적 요구 사항

1. 영상 처리 및 센서값들을 처리하기 위해서 성능이 괜찮은 프로세서 보드가 요구된다. 더불어 저전력 및 고성능 특성, 영상 처리를 위한 별도의 하드웨어 레이어인 Video Process Engine가 있으면 더욱 좋다.
2. 임베디드 보드에 리눅스 운영체제가 설치되어 있어야 한다. 프로세스 관리, 멀티 스레드, IPC(Message queue 그리고 Shared memory), SSH 등의 기능이 필요하다.
3. 효율적인 협동 작업을 위해 git으로 코드의 형상 관리(SCM, Software Configuration Management)를 도입했다. 변경사항을 기록하여 코드의 손실을 예방하고 다른 팀원이 짰 별개 코드의 통합을 수월하게 한다.
4. 팀원들이 각자 나누어 개발한 각 미션의 수행 코드들을 통합할 때, 그리고 로봇이 맵을 주행 할 때 여러 기능들이 충돌하지 않고 의도한 대로 실행되기 위해서 기능들의 의존성을 최소화 하고 소프트웨어 모듈화해야 한다. 모듈화된 각 기능들의 코드는 해석하기 쉬우며 나아가 형상 관리에도 용이하다.
5. 외부 환경에 대한 정보를 받기 위해 CMOS HD급 (1280x720) 카메라가 필요하다.
6. 측면, 전면, 후면에 PSD 센서가 있어야 하며, 하단에 라인 감지용 IR 센서가 필요하다. IR 센서는 차량의 자체적인 calibration 기능을 이용해서 어두운 색 위에 있을 때와 밝은 색 위에 있을 때를 구별할 수 있어야 한다. PSD 센서도 각종 필터를 거쳐서 안정적인 값을 가져오고 거리에 따른 PSD 센서의 값들을 기록하고 curve-fitting하여 (PSD값)-(거리)간 함수를 찾는다.
7. 후륜 DC 모터를 제어할 수 있는 제어보드는 속도제어, 위치제어 두 가지 모드를 제공한다. 각 제어에 필요한 알맞은 상수(PID Gain)를 찾아야 한다. 또한 엔코더 카운트와 거리[cm]의 관계도 찾아야 한다.

○ 개발 방법

1. 개요

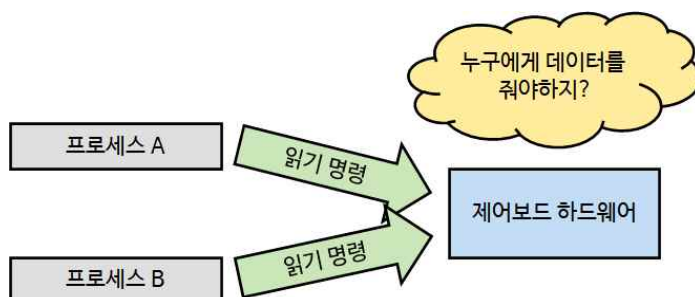


먼저 모형 자동차가 스스로 도로 상황을 판단하기 위해 우선 주변 환경으로부터 유용한 정보를 생산하는 recognize 프로세스를 개발했다. recognize 프로세스에서는 카메라를 통해 받아온 정보를 OpenCV를 이용하여 각종 정보를 만들어 내고 PSD 센서나 IR 센서 데이터를 활용해 모형 자동차 제어에 필요한 정보를 생산한다. 이 프로세스에서 생산한 정보는 리눅스 운영체제의 shared memory 기법을 활용하여 다른 프로세스와 공유할 수 있도록 설계했다.

다음으로 recognize 프로세스에서 생산한 정보를 토대로 주어진 미션을 수행하는 process 프로세스를 개발했다. process 프로세스는 이전에 계획했던 인식-처리-제어의 3단계 모듈화에서 처리와 제어 단계를 합친 것이다. 미션을 수행하기 전에 현재 어떤 state인지 미션 인식을 내부적으로 계속 진행한다. 그리고 미리 정의된 미션 state의 우선순위에 따라 미션을 수행하도록 설계했다.

마지막으로 recognize와 process 프로세스가 동작하기 위해서는 모형 자동차 내부 하드웨어인 제어보드와 UART 통신이 필요하다. 이때 두 프로세스는 서로 분리되어 있고 동시에 제어보드의 자원을 사용해야 하기 때문에 이를 관리해주는 ctrlboard 미들웨어를 개발했다. ctrlboard 미들웨어는 리눅스 운영체제의 message queue를 활용해 다른 프로세스와의 통신을 진행하기 때문에 제어보드에 순차적으로 접근할 수 있도록 해준다.

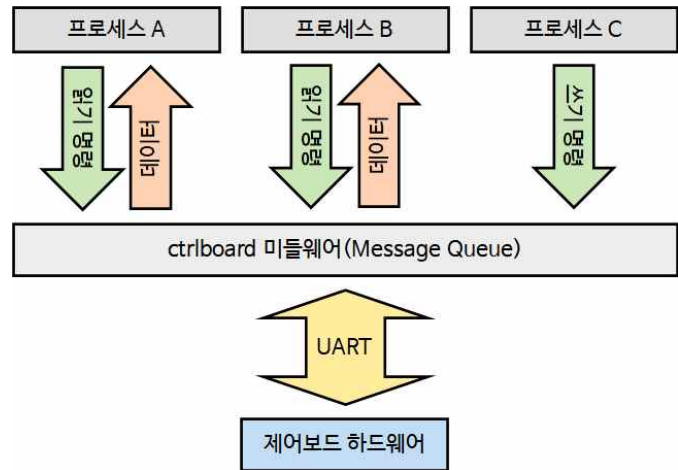
2. 제어보드 통신 미들웨어 개발(ctrlboard)



제어보드란 차량에 부착된 카메라 서보 모터, 바퀴 조향 서보 모터, 전진 및 후진용 모터 등을 제어할 수 있는 하드웨어이다. 또한 IR 센서, 엔코더 데이터 등을 제어보드로부터 받아올 수 있다. 이 제어보드는 UART 통신을 통해 정보를 송수신한다. 이때 제어보드에 여러 프로세스가 UART 통신으로 읽기(Read)

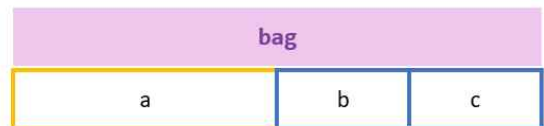
명령을 수행하면 어떤 프로세스가 읽기 명령을 보냈는지 제어보드가 모르기 때문에 읽기 명령의 목적지가 불분명해지는 문제가 생긴다.

이러한 문제를 해결하기 위해 ctrlboard 미들웨어를 따로 만들었다. 그리고 리눅스 운영체제의 message queue를 활용하여 다른 프로세스의 명령들을 ctrlboard 미들웨어의 message queue에 넣도록 했다. 이렇게 받은 message와 message id를 통해 읽기 명령의 결과물을 전달할 위치를 명확하게 지정해주었다. ctrlboard 미들웨어가 일종의 서버 역할을 하며, 제어보드를 사용하려는 클라이언트는 ctrlboard-lib.h를 include해서 ctrlboard 미들웨어와 통신을 수행한다.



```
buf[3] = angle & 0x00ff; //bottom byte
buf[4] = (angle >> 8) & 0x00ff; //top byte
buf[5] = 0xaa + buf[3] + buf[4]; //checksum
write(uart_fd, &buf[0], 6);
```

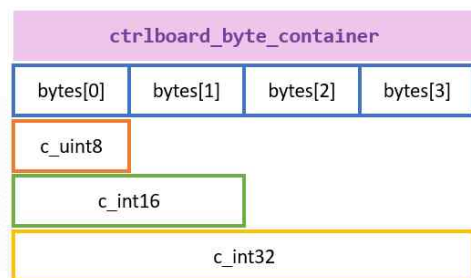
```
struct bag {
    short a;
    char b, c;
}
```



제어 보드와의 통신 함수에서 buf배열 (byte배열)을 사용하지 않고 union을 사용하여 편리성을 더했다. 위의 예제코드의 car_lib.c를 살펴보면 bit shifting하여 변수 angle에 담긴 데이터를 buf 배열에 8비트씩 넣어준다. 그러나 매번 비트연산을 행하는 건 번거롭고 실수의 여지도 있다. 따라서 공용체(union)를 이용하면 별도의 연산 없이 변수들을 byte배열로 접근할 수 있다.

구조체(struct)내의 요소들은 별개의 메모리공간을 차지하지만, union내의 요소들은 아래 그림과 같이 같은 메모리공간을 공유한다.

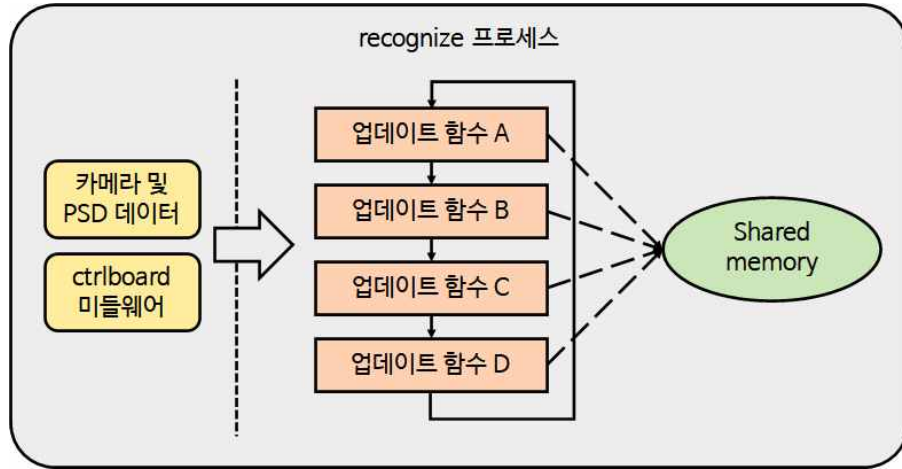
```
typedef union _ctrlboard_byte_container {
    char bytes[MAX_CMD_BYTE_CNT];
    unsigned char c_uint8; // container unsigned 8bit integer
    short c_int16; // container signed 16bit integer
    int c_int32; // container signed 32bit integer
} ctrlboard_byte_container;
```



union 내의 변수들은 같은 메모리 공간을 공유한다. union의 c_int16에 값을 넣으면 이를 비트 연산 없이 바로 bytes 배열을 이용하여 바이트단위로 접근할 수 있다. 아래는 실제 코드 사용 예시이다.

```
short time = 50;
ctrlboard_byte_container data = {time}; // 선언과 동시에 초기화
data.c_int16 = time; // 선언 후 초기화
send_ctrlboard(mqid, CMD_SOUND, 1, &data);
```

3. 인식 프로세스 개발(recognize)



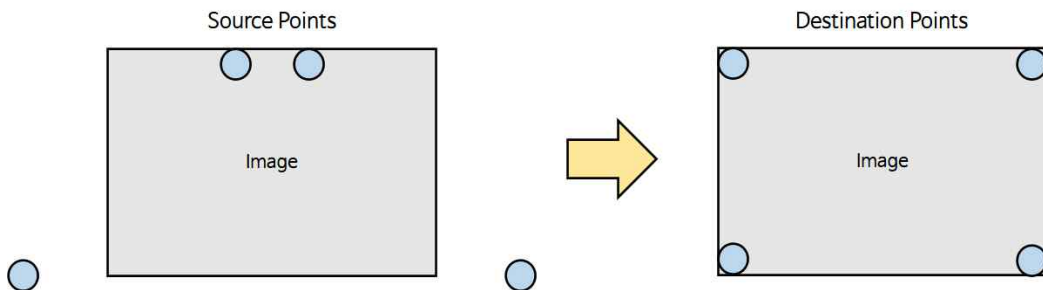
인식 프로세스란 주변 환경으로부터 받은 정보를 사람이 해석할 수 있을 정도의 정보로 가공하는 과정을 수행하는 소프트웨어이다. 이때 생산한 정보를 다른 프로세스들이 공유하기 위해서 리눅스 운영체제의 shared memory 기법을 사용했다. 인식 프로세스에서는 처리 및 제어 프로세스에서 사용할 목적의 정보를 생산하며, 필요한 정보를 최대한 추상화하고 독립적으로 구성해 팀원 각자가 개발할 때 의존성을 최소화하도록 했다. 이때 팀원 각자가 개발하는 인식 프로세스 유닛을 업데이트 함수라고 한다. 자원을 효율적으로 관리하기 위해 업데이트 함수를 켜고 끌 수 있도록 설계했다. 다음은 인식 프로세스에서 인식하는 정보(업데이트 함수)들이다.

(1) 차선의 위치 감지

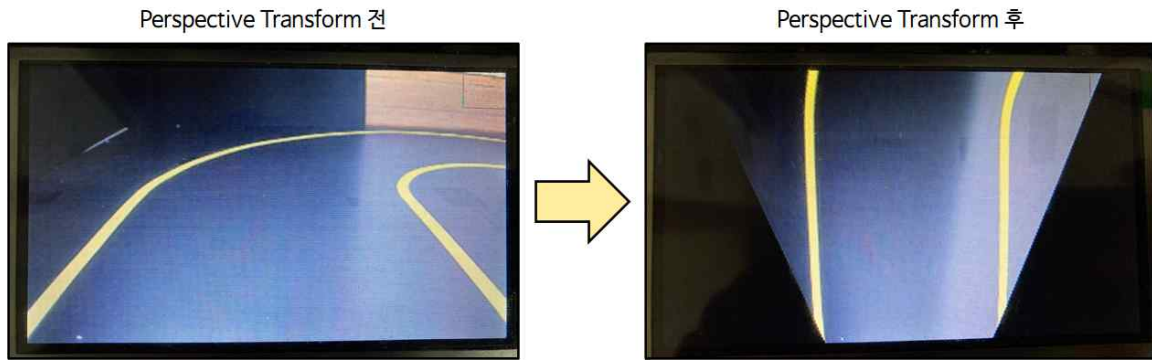
차선의 위치를 구하기 위해 다음 단계를 매 프레임마다 수행한다.

1) Perspective transform 적용하여 Bird's-eye view로 영상 변환

카메라에서 받아온 영상에서는 차선이 실제로 평행하더라도 기울어져 보인다. 따라서 차선을 영상의 y축에 평행하게 하기 위한 Perspective transform 기법을 적용해야 한다. Perspective transform을 적용하기 위해서는 Perspective transform matrix가 필요하다. 이 matrix를 구하기 위해 OpenCV의 getPerspectiveTransform 함수를 사용했다. 이를 이용해 아래의 그림처럼 왼쪽의 네 점을 오른쪽의 네 점의 위치로 변환하는 matrix를 구했다.

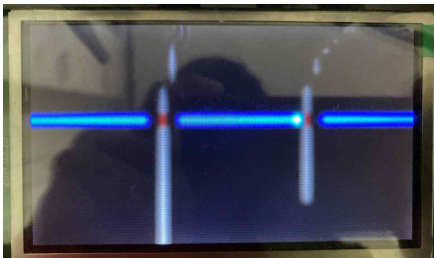


이때 source points는 카메라 각도와 화각으로부터 구했다. 다음으로 카메라 영상의 각 픽셀 별로 perspective transform matrix와 dot product를 수행한다. 이렇게 연산한 결과에 interpolation을 적용해 Bird's-eye view를 구한다. 이때 dot product와 interpolation 과정은 OpenCV의 wrapPerspective 함수를 통해 수행했다. 아래는 위의 과정을 적용하기 전과 후의 사진이다.



2) 영상 크기 축소 및 HSV 색공간으로 변환 후 노란색 차선 영역 추출

영상 크기를 가로 40px 세로 20px 정도로 줄였다. 이는 해상도를 낮추어 잡음을 줄이고 추후 수행할 복잡한 작업들의 수행 시간을 단축하기 위함이다. 그리고 영상을 RGB 색공간에서 HSV 색공간으로 변환한다. 그 후에 H채널과 S채널에 각각 적당한 threshold를 적용해서 이진화 영상을 두 개를 구한다. 그리고 두 영상을 and 연산한다. 이렇게 하면 H채널(색조)에서 차선의 색상인 노란색 영역을 추출하고, S채널(채도)에서 무채색 영역을 걸러낼 수 있다.



그 후 왼쪽의 사진과 같이 노란색 차선만이 검출된 영상에서 적당한 위치의 row 하나를 추출한다(파란색 선). 이렇게 하여 얻어진 1차원 배열은 노란색만을 검출하는 1차원 저해상도 카메라로 촬영한 것으로 생각할 수 있고, 연산 속도를 비약적으로 향상시킬 수 있게 해준다.

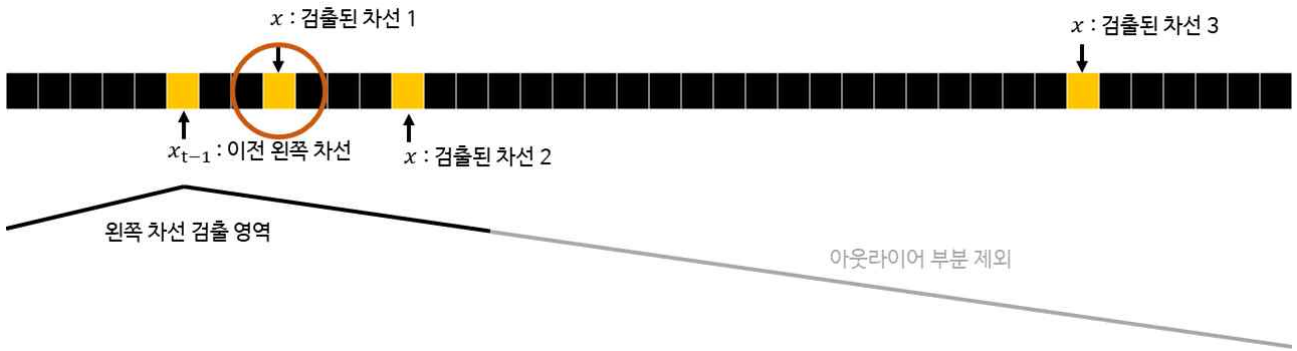
3) 차선 검출

적당한 휴리스틱 함수를 설계하여 해당 row를 이루는 40개의 픽셀 중 노란색 픽셀에 left score와 right score라는 점수를 각각 부여한다. left score가 가장 높은 픽셀의 위치가 왼쪽 차선의 위치가 되고, right score가 가장 높은 픽셀의 위치가 오른쪽 차선의 위치가 된다. left score와 right score를 계산하는 휴리스틱 함수는 화면에 검출된 노란색 픽셀이 우리가 원하는 왼쪽 오른쪽 차선과 얼마나 유사한지 점수를 매기는 함수이다. 현재 사용하는 함수는

$$h(x) = -|x - x_{t-1}|$$

(단, x 는 임의의 픽셀의 x 좌표, x_{t-1} 은 이전 프레임에서 검출된 차선의 위치)

이며, 이는 이전 프레임에서 검출된 차선과 가까울수록 높은 점수를 부여해주는 함수이다. 이 때 이 함수를 왼쪽 차선과 오른쪽 차선에 대해 각각 적용한다.

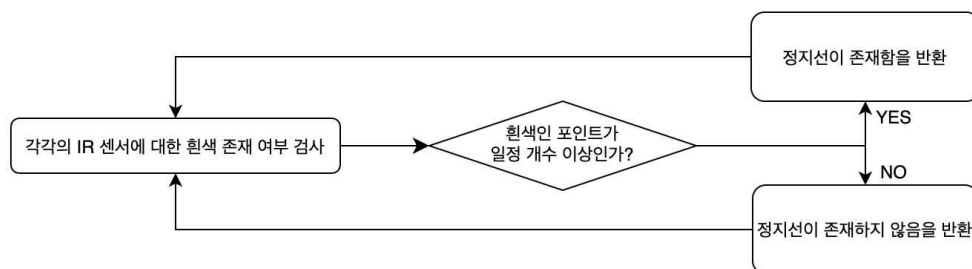


위의 그림은 왼쪽 차선에 대한 휴리스틱 함수를 도식화 한 것이다. 위쪽 영역은 사전 작업한 40개의 픽셀이고 아래쪽 영역은 휴리스틱 함수를 표현한 것이다. 여기서 이전에 검출된 x_{t-1} 위치를 기준으로 가장 가까운 차선인 차선1의 휴리스틱 함수값이 가장 크므로 차선1을 채택한다. 만약, 검출된 차선1과 2가 없을 때 맨 오른쪽에 있는 차선3을 왼쪽 차선으로 채택하게 되는데 이는 실제로는 false positive이므로 휴리스틱 함수의 x 값의 범위를 제한하여 해결한다. 범위 밖에서 검출된 차선에 대해서는 검출되지 않은 것으로 간주한다.

한편, 왼쪽 차선이나 오른쪽 차선 둘 중 하나가 검출되지 않았을 경우, 검출된 나머지 한 차선을 바탕으로 다른 차선의 위치를 추측한다. 그리고 왼쪽 차선과 오른쪽 차선이 모두 검출되지 않았을 경우, 내리막길이나 급한 커브(e.g. S자 커브) 등 특수한 경우로 생각하여 기존 차선 위치를 그대로 유지한다.

(2) 정지선 감지

정지선 감지 업데이트 함수는 자동차의 하단부에 장착되어 있는 IR 센서를 이용하여 정지선을 인식하여 인식 여부값을 반환하는 간단한 함수이다. 회전교차로 구간과 추월차로 구간에서 정지선을 인식하고 멈추기 위해 정지선 감지 알고리즘이 필요하다.



자동차의 하단부에 장착되어 있는 IR 센서를 이용하여 일정 주기마다 IR 센서의 값을 불러온다. 자동차에서 IR 센서의 값은 8비트 변수에 1또는 0의 형태로 저장된다. 자동차가 내부적으로 기준치보다 높은 아날로그 값을 받게되면 디지털 값인 0을 제어보드로부터 받고 반대로 기준치보다 낮은 아날로그 값을 받게되면 디지털 값인 1을 제어보드로부터 받는다.

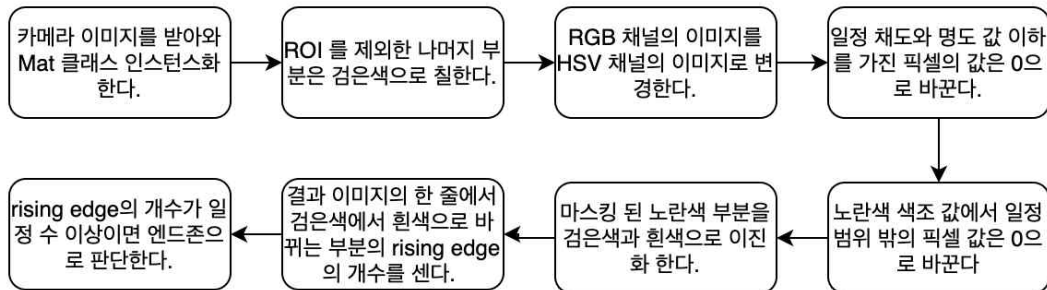
또한 IR센서는 7개 인데 반해 해당 정보가 담기는 변수는 8비트 변수이므로 한 비트는 쓰지 않게 되는데, 해당 변수의 MSB는 항상 0으로 고정되어 있는 것을 테스트를 통해 발견하였다. 테스트를 진행하지 않았을 때는 1이 흰색을 나타내고 0이 검은색을 나타낼 것으로 예상하였는데, 반대로 0이 흰색을 나타내고 1이 검은색을 나타내는 것이 특이점이었다.

우리가 작성한 정지선 검출 알고리즘은 하단 IR 센서의 흰색이 검출 개수가 특정 개수 이상

이 되면 정지선임을 반환하는 것이 특징이다.

(3) 종료지점 감지

이 업데이트 함수는 종료 지점을 영상처리를 통해 인식하고 존재 여부를 반환하는 함수이다. 모든 미션을 마치고 종료 지점에 진입할 때, 종료 지점 안에서 잘 멈추기 위해 이 인식 함수가 필요하다.

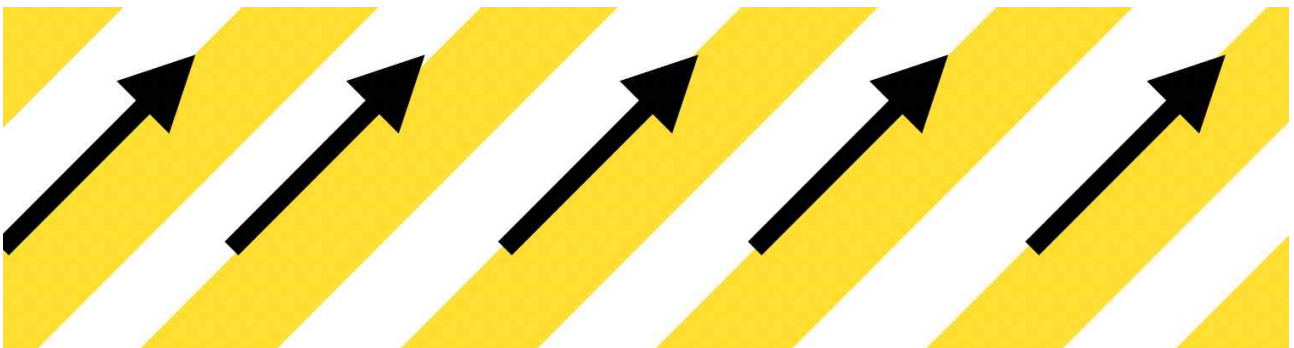


OpenCV의 클래스와 함수들을 이용하여 종료지점을 인식하여 존재 여부를 반환하는 알고리즘을 작성하였다. 받아온 영상 데이터는 1차원 배열의 형태로 저장된다. 이때 OpenCV의 Mat 클래스를 사용하면 2차원 배열의 형태로 저장되고, RGB색공간을 이용하여 저장된 이미지는 3개의 채널을 가진 2차원 배열의 형태로 저장된다.

변환한 Mat 클래스 인스턴스에서 관심영역(Region of Interest) 을 제외한 나머지 부분은 우선 검은색으로 칠한다. 본 팀에서 작성한 코드에서는 우선 자동차의 카메라 서보 모터 각을 가능한 아래로 내렸다. 그 다음 아래부터 전체 화면의 1/4 정도를 Roi로 설정하였다.

Roi가 적용된 이미지를 OpenCV를 이용하여 HSV 색공간 영역의 이미지로 변경한다. 이후 노이즈를 없애기 위해 일정 채도와 명도 값 이하를 가진 픽셀의 값을 검은색으로 바꾼다. 모든 픽셀의 H값을 검사하여 노란색으로 생각되는 색조의 범위 밖의 픽셀 값을 검은색으로 바꾼다.

위의 작업을 거치게 되면 이미지의 픽셀들 값을 보았을 때 노란색 이미지들만 값을 가지고 나머지는 검은색, 즉 0의 값을 가지게 된다. 따라서 값을 가진 부분들은 모두 하얀색으로 바꿔주는 이진화 작업을 하였다.



이진화 작업이 마무리된 이미지의 한 row 에 대해 검은색 값에서 흰색 값으로 바뀌는 rising edge 의 개수를 센다. rising edge 의 개수가 일정 수 이상이 되면 종료지점으로 판단한다.

한편, 이미지를 받아와서 threshold 함수만을 사용하였을 때는 외부 요인에 의해 그림자나 노

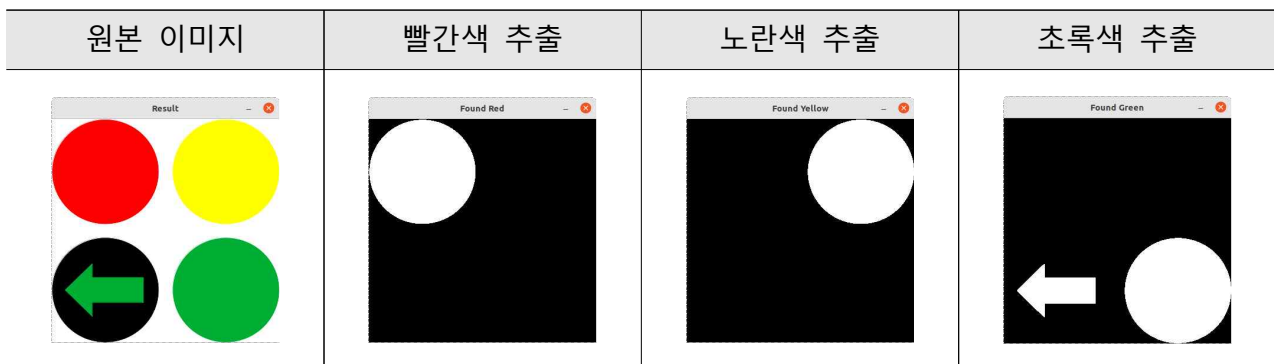
이즈가 생겼을 때 종료 지점 인식을 못하는 경우가 많았다. 그래서 adaptiveThreshold 함수의 전체 평균법을 사용했더니 외부 요인의 영향을 받는 일이 매우 적어졌고 인식률이 높아졌다.

(4) 신호등 정보

신호등 정보를 인식하기 위해 OpenCV 라이브러리로 특정 색상을 가진 도형을 추출하는 작업을 하였다. 매 프레임마다, 아래의 작업을 순차적으로 실행하여 현재 차량의 카메라에 보이는 신호를 인식하였다.

1) 포착한 이미지에서 빨간색, 노란색, 초록색 부분만을 각각 추출하여, 0 또는 1로 이루어진 마스크 행렬들을 만든다.

이미지에서 각 색상을 추출하여 빨간색을 인식한 마스크, 노란색을 인식한 마스크, 초록색을 인식한 마스크로 총 세 가지의 마스크 행렬을 만든다. 마스크를 만들 때, RGB 형식의 이미지를 HSV 색공간으로 변환한 뒤, 원하는 색상이 가지는 H 값의 구간에 포함되는 픽셀 만을 추출한다. 이 때, S값과 V값이 너무 작을 경우 단순히 흰 색과 어두운 색을 원하는 색으로 인식하게 되는 문제가 있으므로, 인식할 S값과 V값의 최솟값을 정의해줌으로써 보다 정확도 높게 색상을 추출할 수 있다.



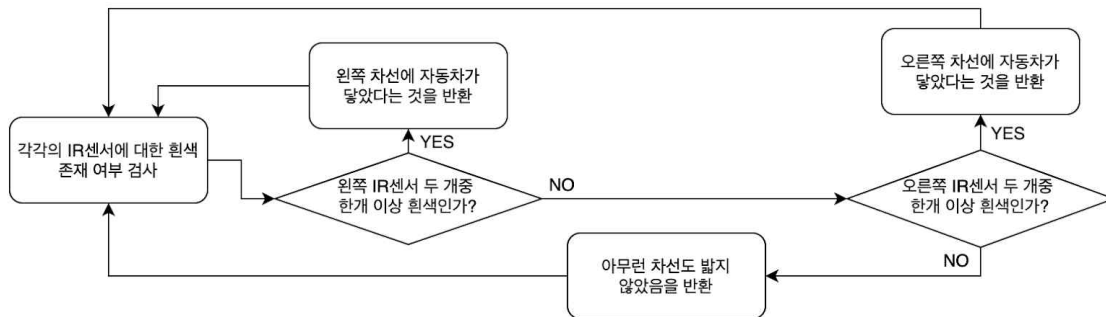
2) 이전에 만든 마스크에서 1로 이루어진 영역 중 원, 왼쪽 화살표, 오른쪽 화살표를 각각 인식한다.

마스크에서 원과 화살표를 인식하는 것은 세부적으로 보았을 때 아래와 같은 과정을 따른다.

- 마스크에서 1로 이루어진 영역의 윤곽선을 추출한다. 이를 위해 OpenCV의 findContours 함수를 사용할 수 있다.
- 찾아낸 윤곽선들 각각에 대하여, 그 윤곽이 포함하는 영역의 넓이가 특정 범위에 속하는 지 확인한다. 이를 확인하는 이유는 영상에서 노이즈나 배경에 나타난 색상을 잘못 인식하는 것을 방지하기 위함이다.
- Douglas-Peucker algorithm을 이용하여 윤곽을 다각형으로 근사시킨다. 이는 OpenCV의 approxPolyDP 함수를 이용하여 근사할 수 있다.
- 근사한 다각형이 Convex한지와 다각형을 이루는 변의 개수에 따라 그 다각형이 어떤 도형인지를 판별한다.
 - a) 도형이 Convex하지 않으면서, 변이 7개라면, 이를 화살표로 인식한다. 여기에서 그 화살표가 왼쪽을 가리키는지, 오른쪽을 가리키는지는 도형의 꼭지점이 왼쪽에 더 많은지, 오른쪽에 더 많은지를 판단하여 결정할 수 있다.
 - b) 도형이 Convex하면서, 변이 7개보다 많다면, 이를 특정 색의 원 모양의 신호로 인식한다.

(5) 차선을 밟았는지의 여부

이 업데이트 함수는 자동차를 기준으로 왼쪽 차선이나 오른쪽 차선에 자동차가 닿으면 이를 인식하고 반환하는 함수이다. 주행 중 자동차가 예기치 않게 차선을 벗어나려는 상황에서 이를 감지하고 다시 정상적인 주행 궤도로 돌아가기 위해서 필요하다.



주행 중 자동차가 예기치 않게 차선을 벗어나는 상황을 막고자 해당 함수를 만들게 되었다. 이 알고리즘은 정지선 검출과 마찬가지로 자동차 하단부의 IR 센서를 매우 빠른 주기로 반복적으로 검사하여 바닥에 흰색 선이 있는지 검출한다.

왼쪽 두 개 IR센서의 어느 한 부분이라도 흰색이 검출되면 자동차가 왼쪽 차선을 침범했다는 값을 반환하고, 마찬가지로 오른쪽 두 개 IR센서의 어느 한 부분이라도 흰색이 검출되면 자동차가 오른쪽 차선을 침범했다는 값을 반환한다.

(6) 내리막길(경사로)에 있는지 여부

이 업데이트 함수는 전방에 내리막길(경사로)가 있는지 인식하고 반환하는 함수이다. 이 정보가 필요한 이유는 경사로 구간 즉, 내리막길을 인식하지 않고 속도를 늦추지 않은 채 내리막길을 주행하면 경사로에서 미끄러져 주행이 제대로 되지 않기 때문이다. 내리막길을 파악하기 위해 다음 과정을 매 프레임마다 수행한다.

1) HSV 색공간으로 변환 후 노란색 차선 추출

OpenCV의 `cvtColor` 함수를 이용해서 RGB 색공간을 HSV 색공간으로 변환한다. 그리고 적당한 노란색 영역을 설정한 뒤 `inRange` 함수를 통해 일정 범위의 색상만 뽑아낸다. `inRange` 함수는 두 `Scalar` 인스턴스 사이의 값에 해당하는 픽셀만 255로 설정해주는 binarization을 진행한다.

2) 마스킹 이미지 적용(RoI 설정)

미션이 고가도로에서 내리막으로 진행하기 때문에 내리막길 진입 전 전방에 보이는 수많은 차선들이 보인다. 우리의 목표는 양쪽에 있는 총 두개의 차선만 인식해 교점을 구하는 것인데, 다른 차선들이 보여서는 안된다. 따라서 RoI를 적절히 설정해야 한다.



위의 그림의 첫 번째 사진과 두 번째 그림을 AND 연산해 세 번째 사진의 결과물을 도출한다. 즉, 두 번째 영상의 하얀 부분(255) 값과 첫 번째 영상과 AND 연산을 하기 때문에 좌우의 차선만 남길 수 있다. 이렇게 되면 외부 간섭을 없앨 뿐만 아니라 직선을 구하는데 용이해진다.

3) Hough Transform을 통한 직선 검출

Hough Transform으로 직선을 검출하여 RoI에 보이는 직선을 검출한다. 이때 사용한 함수는 OpenCV의 HoughLinesP 함수이며, 이는 직선의 시작점과 끝점을 반환한다. 이때 여러 직선들 중 왼쪽 RoI에서 가장 아래쪽, 오른쪽 RoI에서 가장 아래쪽에 있는 직선만 추출한다.

4) 검출된 두 직선의 교점 구하기

검출된 두 직선의 교점을 구하고 첫 번째, 이 교점의 y값이 0보다 크고 두 번째, 두 직선의 기울기의 곱이 음수일 경우에 기울기로 판단한다. 교점의 y값이 0보다 클 때 기준으로 한 이유는 내리막길에 진입할 때 두 차선의 기울기가 거의 0에 수렴하기 때문이다. 한편, 일반적인 도로주행 상황에서는 차선이 기울어져있고 거의 화면 위쪽에 차선이 보이기 때문에 차선의 교점의 y좌표가 0보다 작아진다. 따라서 일반적인 도로 주행과 경사로를 혼동할 일이 생기지 않는다. 하지만 커브길에서 내리막임을 감지할 때가 있었는데 두 직선의 기울기의 곱이 음수일 경우에만 기울기를 판단하도록 조건을 좁혀 해결했다.



(7) 터널 감지

차량 앞쪽에 있는 PSD 센서 두 개를 이용하여 터널을 감지할 수 있다. 앞쪽의 두 PSD 센서가 일정 거리 이하이면 터널의 양쪽 벽을 감지했다는 의미이므로 터널 구간임을 알 수 있다.

(8) 우선 정지 장애물 감지

우선 정지 장애물을 감지하는 것은 이전에 신호등을 감지하는 과정과 매우 유사하다. 그 이유는 우선 정지 장애물을 인식하는 것 역시 카메라로 포착한 이미지에 대하여 영상 처리를 이용하여 우선 정지 장애물의 패턴을 인식하는 것이기 때문이다.

신호등 정보를 인식하기 위해 OpenCV 라이브러리로 빨간색을 가진 직사각형과 원형 도형을 추출하는 작업을 하였다. 매 프레임마다, 아래의 작업을 순차적으로 실행하여 현재 차량의 전방에 우선 정지 장애물이 존재하는지를 인식하였다.

반환하는 값으로 우선 정지 장애물의 위치값(x, y)와 면적을 반환한다. 면적을 통해 우선 정지 장애물이 얼마나 가까운지 판단할 수 있다.

1) 포착한 이미지에서 빨간색 부분만을 각각 추출하여, 0 또는 1로 이루어진 마스크 행렬을 만든다.

마스크를 만들 때, RGB 형식의 이미지를 HSV 색공간으로 변환한 뒤, 붉은 색에 해당하는 H 값을 가지는 픽셀 만을 추출한다. 이때, S값과 V값이 너무 작을 경우 단순히 흰 색과 어두운 색에서 H값만 만족해도 붉은색으로 인식하게 되는 문제가 있으므로, 인식할 S값과 V값의 최소값을 정의해줌으로써 보다 정확도 높게 색상을 추출할 수 있다.

2) 추출한 마스크에 대하여 사각형과 원을 찾는다.

사각형과 원을 찾는 것은 이전에 소개한 신호등을 감지할 때 특정 도형을 찾는 과정과 매우 유사하고, 실제 코드에서도 서로 같은 함수를 사용하고 있다. 이 과정을 세부적으로 설명하면 아래와 같다.

- 마스크에서 1로 이루어진 영역의 윤곽선을 추출한다. 이를 위해 OpenCV의 findContours 함수를 사용할 수 있다.
- 찾아낸 윤곽선들 각각에 대하여, 그 윤곽이 포함하는 영역의 넓이가 특정 범위에 속하는 지 확인한다. 이를 확인하는 이유는 영상에서 노이즈나 배경에 나타난 색상을 잘못 인식하는 것을 방지하기 위함이다.
- Douglas-Peucker algorithm을 이용하여 윤곽을 다각형으로 근사시킨다. 이는 OpenCV의 approxPolyDP 함수를 이용하여 근사할 수 있다.
- Convex한 다각형에 대하여, 변의 개수를 확인하여 원과 사각형을 구별한다. 변의 개수가 4개라면 사각형, 변의 개수가 7개 이상이라면 원으로 인식한다.

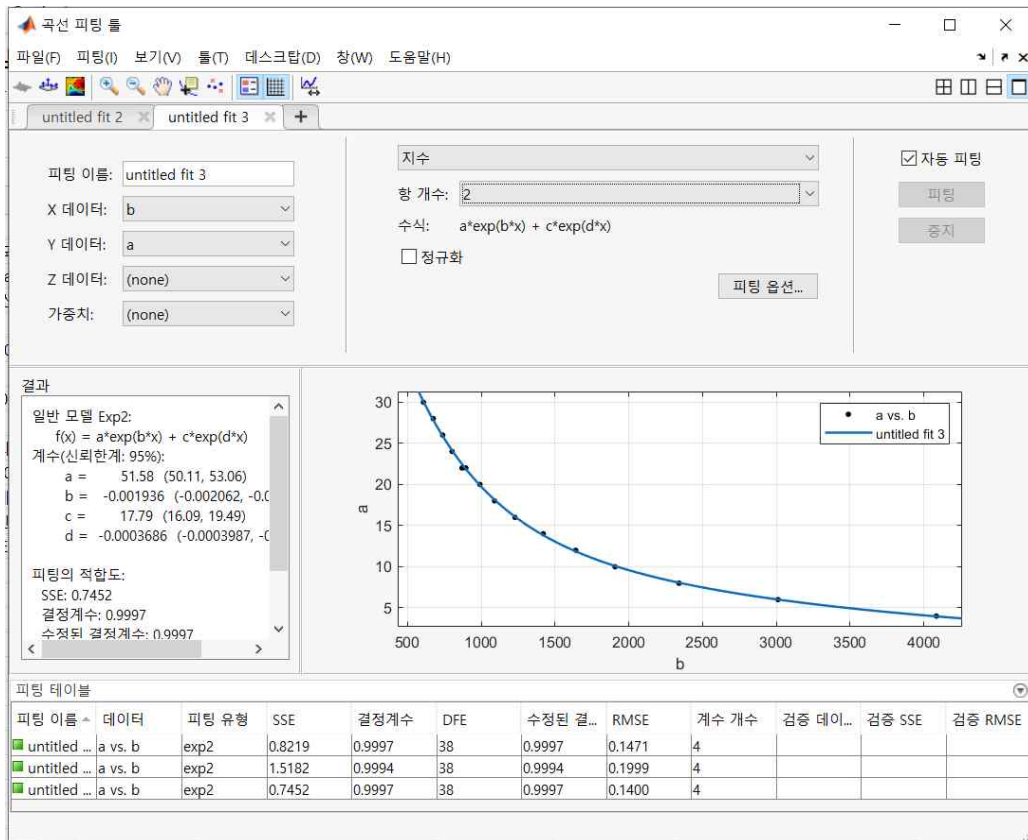
3) 인식한 원과 사각형에 대하여 우선 정지 장애물의 위치관계가 성립하는 조합이 존재하는지 확인한다.

우선 정지 장애물은 하나의 직사각형 내부에 선이 흰색인 원 하나가 존재하는 패턴이다. 여기에서 흰색과 빨간색의 경계를 도형의 윤곽으로 취급한다면, 이는 직사각형 내부에 원이 있고, 그 원 내부에 또 다른 원이 있는 형태라고 말할 수 있다. 이러한 위치 관계가 성립하는지 판별하기 위해 OpenCV의 pointPolygonTest 함수를 이용하여 특정 도형이 다른 도형의 내부에 존재하는지 판별할 수 있다.

(9) PSD 센서값

로봇의 앞, 뒤 그리고 양 옆에 총 6개의 PSD가 존재한다. 로봇과 각종 장애물과의 거리를 PSD를 통해 알 수 있다. 각 센서를 거리마다 테스트하여 센서값을 측정하고 MATLAB의 cftool을 이용하여 거리를 센서값에 관한 함수로 나타낸다. $distance = f(data)$ 제어보드에서 센서값을 읽어 오면 이를 cm단위의 거리로 변환하여 recognize result 구조체의 psd항목을 업데이트한다. 고가도로나 터널과 같은 벽을 따라 주행하는 구간이나 주차와 같은 구조물을 이용하여

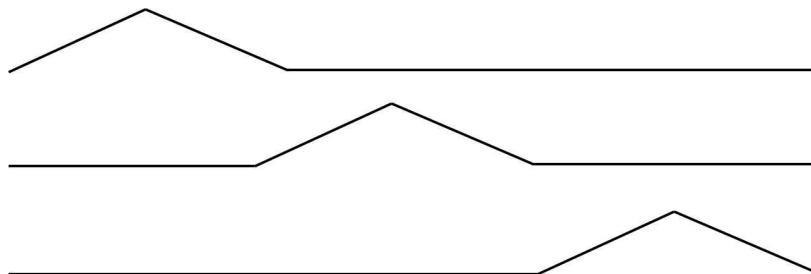
미션을 수행하는데 사용된다. 센서의 datasheet에 따르면 약 4~30cm거리의 장애물을 파악할 수 있으므로 $\text{distance} = f(\text{data})$ 함수를 통해 얻은 distance가 4cm미만이거나 30cm를 초과하지 않도록 한다. 센서에 노이즈가 발생하므로 센서에서 3개의 데이터를 얻어서 그 중앙값을 distance로 변환한다.



위의 그림은 로봇의 오른쪽앞에 위치한 PSD의 (센서값)-(거리) 함수의 모습이다. 지수함수의 모습을 띄고 있어서 지수함수의 항 2개를 사용하여 $\text{distance} = f(\text{data})$ 함수를 만들었다.

(10) 차로에 물체가 있는지 검사

추월 차로 미션을 수행하기 위해 세 개의 차로 중에 어떤 차로에 물체가 존재하는지 인식할 필요가 있다. 이를 인식하기 위해서 먼저 영상을 그레이스케일로 변환한다. 차로는 검은색이고 물체는 검은색이 아니기 때문에 영상을 그레이스케일로 변환하면 물체를 더욱 쉽게 인식할 수 있다. 이렇게 그레이스케일로 변환한 영상과 특정 함수를 영상의 각 row에 대해 dot product를 취하고 결과값들을 모두 더한 값을 구한다. 특정 함수는 총 3개이며 다음과 같다.



위의 함수들 중 첫 번째 함수는 영상을 3등분 했을 때 가장 왼쪽에 있는 픽셀들의 가중치를 주는 함수라고 볼 수 있다. 따라서 이 함수를 적용했을 때 일정 수준 이상의 값이 나온다면 가

장 왼쪽 차선에 물체가 존재한다고 판단할 수 있다. 나머지 두번째와 세번째 함수도 마찬가지로 위의 의미를 지닌다.

한편, 계산 복잡도와 인식 정확도를 높이기 위해 중심으로부터 일정 수준의 양 옆이 0인 절댓값 함수를 사용하는데 이는 다른 함수(예를 들어 이차함수)에 비해 연산량이 적게 들고, 위로 볼록한 함수를 사용함으로써 차로 양옆에 있는 차선에 대한 가중치를 줄여 더욱 정확한 차량 여부를 알아낼 수 있다. 더불어 극대점으로부터 일정 수준의 양 옆이 0이기 때문에 다른 차로의 영향은 받지 않게 할 수 있다.

이렇게 차로에 물체가 있는지 여부를 판단한 후에 공유 메모리에 차로의 왼쪽, 중앙, 오른쪽에 물체가 있는지 bool값으로 저장한다.

4. 처리 및 제어 프로세스 개발(process)

process 프로세스는 recognize 프로세스로부터 받은 정보를 취합하여 어떤 동작을 할지를 결정하는 소프트웨어이다. 이때 동작이란 두 가지로 나뉘는데, 하나는 recognize 프로세스의 제어이며 나머지는 하드웨어의 제어이다. recognize 프로세스의 제어란 recognize 프로세스에서 수행하는 특정 과정을 켜거나 끄는 것을 말한다. 예컨대 출발 후에는 출발 신호 인식 기능을 아예 꺼서 오작동을 막고 효율적으로 CPU를 사용할 수 있도록 한다. 하드웨어의 제어란 말 그대로 제어보드를 통하여 하드웨어를 제어하는 것으로, recognize 단계에서 얻은 여러 정보를 바탕으로 어떤 동작을 해야할지 결정한 후 ctrlboard 미들웨어로 적절한 하드웨어 명령어를 보낸다. 가장 간단한 예는 조향으로, 인식 단계에서 차선의 위치를 전달받은 후 PI 제어를 사용해 적절한 차량 앞바퀴 조향 각도를 계산, ctrlboard 미들웨어로 이를 전송한다.

그런데 입력에 따라 동작하는 것은 간단하게 이루어지는 것이 아니라, 미션에 따라 다르게 이루어져야 한다. 그리고 이 미션들을 서로 독립적으로 개발 및 구동이 가능해야 한다. 따라서 다음과 같이 동작하도록 구성하였다.

1. 각 미션은 독립적인 파일로 구현된다.
2. 각 파일은 `init_{missionName}`, `check_{missionName}`, `do_{missionName}`의 세 함수를 구현한다. (필요에 따라 `init`을 구현하지 않을 수도 있다.)
3. `init`함수는 단 한 번만 실행되며, 구조체 초기화 등 미션 수행에 초기화 동작들을 수행한다.
4. `check`함수는 어떤 미션이 수행되어야 하는지 그 여부와 우선순위를 반환한다.
5. `do`함수는 실제로 미션을 수행한다. 만약 어떤 미션 수행 중 다른 미션이 나올 수 있는 경우라면 `do` 함수는 한 동작만을 수행하고 바로 반환해야 하며, 그렇지 않고 반드시 모든 동작을 한 번에 수행해야 하는 경우 `while`문 등을 사용하여 모든 동작을 수행한 후 반환해야 한다. 예를 들어 주행을 하던 도중에는 여러 미션이 나올 수 있기 때문에, 한 동작만을 수행하고 바로 반환한다. 그러나 주차 미션을 수행하는 중에는 그럴 염려가 없으므로 모든 동작을 수행하고 반환한다.
6. `mission` 프로세스는 각 `check`함수를 돌면서 어떤 미션을 수행해야 하는지를 체크한다.

선택된 미션들 중 가장 우선순위가 높은 미션이 수행된다.

(1) 도로주행

도로주행은 오직 recognize 프로세스의 차선 위치 정보 하나만을 사용한다. 차선 정보에 대해 단순 P제어를 사용하여 주행한다.

(2) 고가도로 구간

고가도로에는 도로 양 옆에 얇은 벽이 있다. 양 옆 PSD를 사용하면 거리정보만 이용해서 주행할 수 있다. 도로의 폭은 약 40cm이고 로봇이 도로 중앙에 위치할 때 측면 PSD는 약 10cm의 거리를 반환한다. 왼쪽과 오른쪽 PSD의 거리의 차이를 이용하여 도로에서 로봇이 어느쪽으로 치우쳤는지 알 수 있다.



왼쪽 PSD값에 오른쪽 PSD 값을 뺀 D값과 상수 gain_p를 곱하여 앞 바퀴 조향각을 결정한다.

(3) 돌발(내리막 구간)

돌발(내리막 구간)에는 오직 recognize 프로세스의 내리막길(경사로) 인식 값만을 이용한다. recognize 프로세스의 내리막길(경사로) 인식 값이 참일 경우에 속도를 낮추어 돌발(내리막 구간)에 진입하고, 경사로를 빠져나올 때 다시 속도를 복구한다. 경사로를 빠져나오는 조건은 앞쪽의 두 PSD 값이 사전에 정의된 최대치를 달성하면 경사로를 빠져나오는 조건으로 했다.

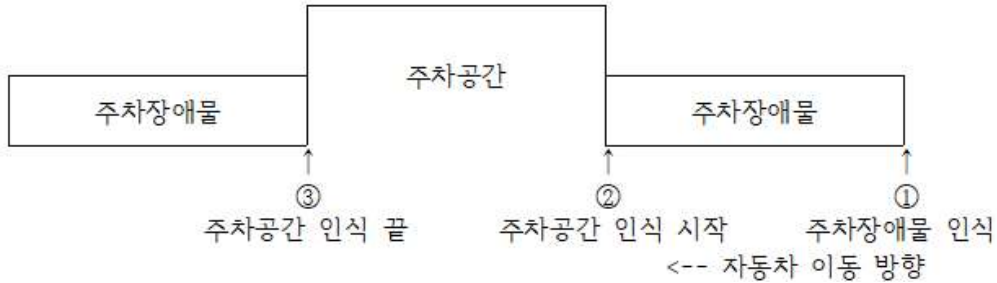
(4) 우선정지 장애물 구간

우선정지 장애물 구간에서는 오직 recognize 프로세스의 우선 정지 장애물 감지 정보만을 이용한다. 우선 정지 장애물의 면적값이 일정 수준 이상이면 우선정지 장애물 구간으로 인식하여 멈춘 다음에 우선 정지 장애물의 면적값이 0이 될 때까지 기다린다. 값이 0이 된다면 다시 일반적인 도로 주행으로 돌아간다. 한편, 우선 정지 장애물을 잠깐 동안 인식하지 못했을 때 바로 일반적인 도로 주행으로 들어가는 것을 방지하기 위해 one shot 필터를 적용했다.

(5) 곡선 S자 주행

일반 도로주행과 마찬가지로 곡선 S자 주행 미션을 수행한다.

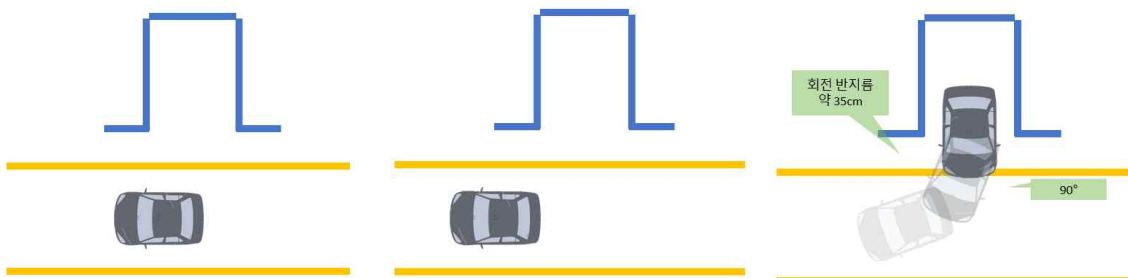
(6) 수평 및 수직 주차



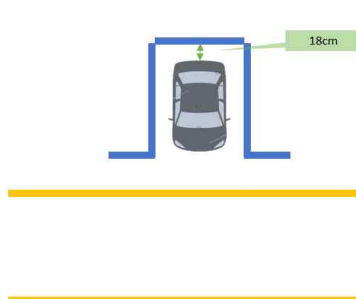
측면 앞에 위치한 PSD로 ①의 주차장애물을 인식하고 나서 PSD가 인식한 거리가 29cm이상 멀어지면 ②의 주차공간을 인식하기 시작한 것으로 판단한다. 이 때의 엔코더 카운터의 값을 x 라고 하자. 다시 PSD가 주차장애물을 인식해서 ③주차공간의 끝으로 판단하였을 때의 엔코더 카운터의 값을 y 라고 했을 때 ②와 ③ 사이의 엔코더 카운터 값의 차인 $(y-x)$ 을 이용하여 주차공간의 너비를 구할 수 있다. 로봇이 1cm가는 데 약 19.7628의 엔코더 카운터가 증가하므로 인식한 주차공간의 너비는 $(y-x)/19.7628$ [cm]이다. 이 값이 30이상 40미만이면 수직주차 구간이고, 50이상 60미만이면 수평주차구간이다.

위의 과정을 통해 주차공간을 인식하면 바로 주차미션으로 진입하며 로봇을 멈춘다.

a) 수직 주차

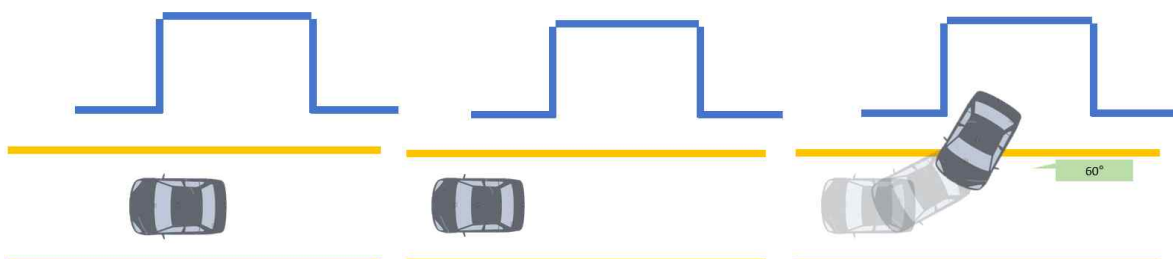


자동차가 주차구간을 인식하여 수직 주차 미션에 진입한다. 멈춘 뒤 적절한 위치로 전진한다. 그 다음 조향을 최대로 틀어서 90도 회전한다.

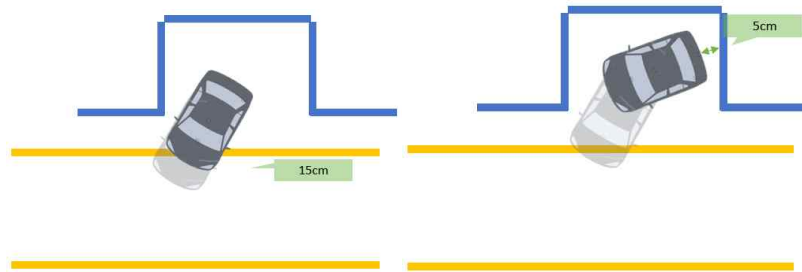


뒤쪽 PSD와 벽과의 거리가 18cm가 될 때까지 후진하면 주차 성공이다. 위의 과정을 반대로 행해서 도로주행하기 적합한 상태로 복귀한다.

b) 수평 주차



수평 주차 미션에 진입한 다음 멈춘다. 적절한 위치로 전진한 다음 조향을 오른쪽으로 최대로 틀어서 60도 회전한다.



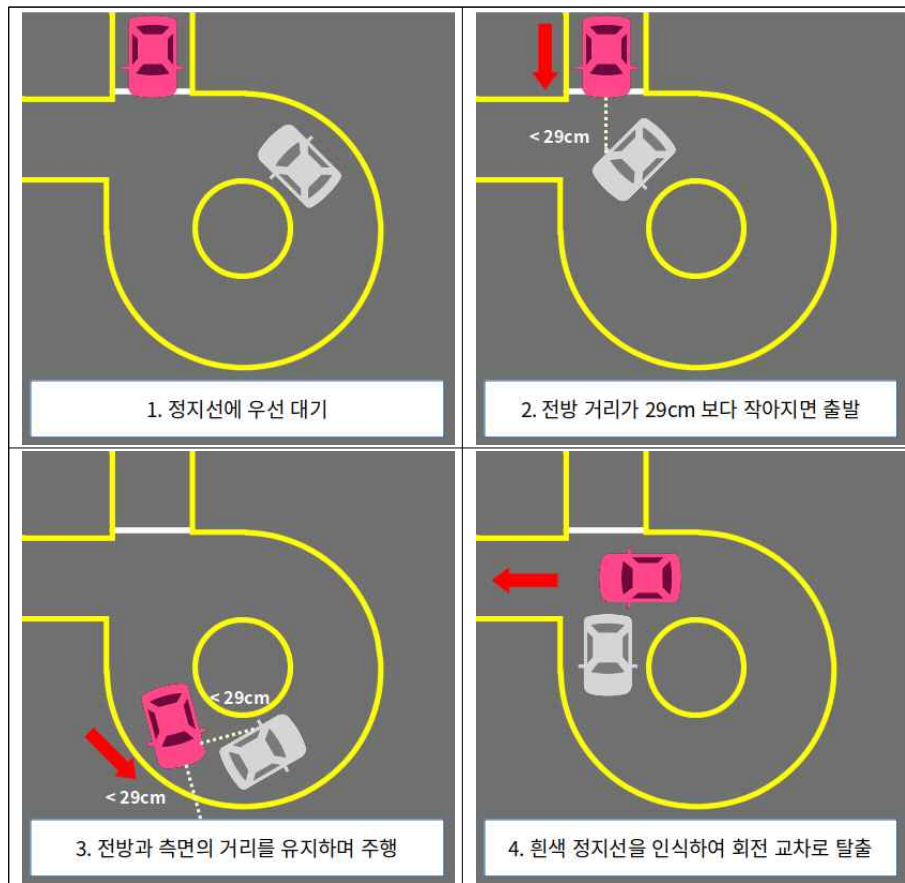
다시 조향을 중앙으로 맞추고 15cm후진한다. 조향을 왼쪽으로 최대로 틀어서 후진한다. 뒤쪽 PSD와 벽과의 거리가 5cm가 될 때까지 후진하면 주차성공이다. 위의 과정을 반대로 행해서 도로주행하기 적합한 상태로 복귀한다.

(7) 회전 교차로

회전 교차로를 주행하는 것은 이전에 소개한 도로 주행 미션을 진행하는 것과 유사하다. 즉, 원형의 도로를 앞 차와의 일정 거리를 유지하면서 주행하고, 기존의 도로 주행 미션과 다르게 노란색 차선 뿐만 아니라 흰색 차선 또한 인식하여 회전 교차로를 안정적으로 탈출할 수 있다. 이 과정을 구체적으로 살펴보면 아래와 같다.

1. 하단의 IR 센서를 통해 회전 교차로의 정지선을 인식하여 정지한다.
2. 회전중인 장애물이 차량의 앞에 올 때까지 기다리기 위해, 전방의 PSD 센서 값이 29 아래로 떨어지는 것을 기다린다.
3. 전방의 PSD 값이 29 아래로 떨어진다면 출발하여 회전 교차로를 주행하면서 반복적으로 PSD 센서의 값을 확인한다. 전방 PSD 센서의 값과 왼쪽 측면의 차량 앞 쪽에 있는 PSD 센서의 값이 29 아래가 된다면 속도를 줄여 앞 장애물과의 간격을 유지한다.

여기에서, 회전 교차로를 주행하는데 도로 주행 미션을 수행하는 코드를 활용한다. 일반적으로 도로를 주행하는 원리를 그대로 활용하되, 노란색 차선 뿐만 아니라 흰색 차선 또한 감지하도록 한다. 이렇게 하면 차량이 회전 교차로를 한 번 돌았을 때, 자신의 오른쪽에 있는 흰색 정지선을 앞쪽으로 뺀 도로로 인식할 수 있으므로 자신이 전방으로 가야 함을 인식할 수 있다.



그리고 회전 교차로 미션 State를 종료 시키는 조건은 전방에 흰색 정지선이 감지되는지를 확인하여 판단할 수 있다. 회전 교차로를 완전히 돌고 나면 자신이 들어올 때 지난 흰색 정지선을 다시 감지하게 되므로 이를 영상처리로 감지하여 직선 주행을 해야 하는 것을 차량이 판단할 수 있다. 이전처럼 측면과 전방의 PSD 센서를 이용해 앞 장애물과의 거리를 유지하며 이동하다가, 1초간 장애물이 감지되지 않는다면 회전 교차로 미션 State를 종료하고 일반적인 도로 주행 미션으로 돌아간다.

(8) 터널 코스

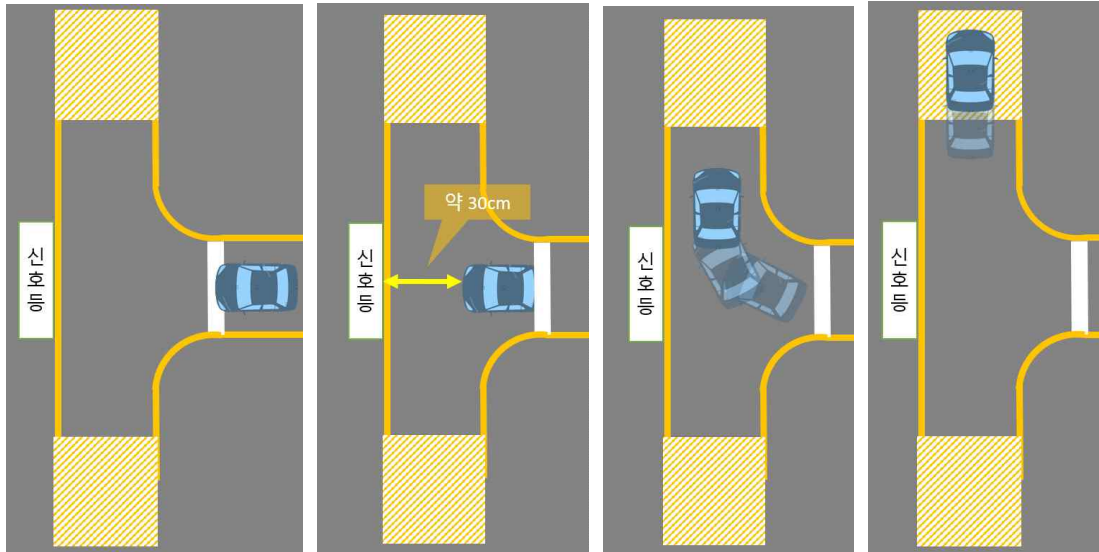
고가도로 구간에서 했던 것처럼 차량의 양쪽 PSD를 통해 터널 왼쪽과 오른쪽 벽과의 거리가 같게끔 조향하여 주행한다.

(9) 차로 추월 구간

1. 일반적인 주행 중 차량의 앞쪽 PSD센서를 이용하여 일정 거리 이하로 떨어지면 앞질러야 할 차가 있음을 인지한다.
2. 일정 거리를 뒤로 이동해 카메라 틸트를 높인다.
3. 차로위의 물체 존재 여부를 recognize 프로세스에서 가져온다.
4. 왼쪽 차로가 비어있다면 왼쪽으로 추월한다. 오른쪽 차로가 비어있다면 오른쪽으로 추월한다.
5. 추월 후 다시 일반적인 도로 주행으로 돌아가 직진한다.

(10) 신호등 분기점 코스 및 종료 지점

정지선을 발견하면 멈추고 신호등이 지시하는 방향으로 이동한다. 인식 프로세스에서 정지선과 신호등, 그리고 종료 지점의 정보를 가져와서 사용한다.



IR센서로 정지선을 발견하면 즉각 멈춘다. 카메라 틸트 각도를 수평에 맞추어서 신호등을 바라본다. 좌회전 신호 또는 초록불을 인지하면 앞쪽 PSD가 약 30cm를 볼 때까지 전진한다. 그 후 신호등이 지시하는 방향으로 조향을 최대로 틀어서 90도 회전하면 종료 지점에 바르게 들어갈 수 있다. 카메라 틸트 각도를 다시 내리고 recognize 프로세스의 종료 지점 여부 값을 검사한다. 이 값이 true이면 계속 전진하고, false가 되면 멈추어 미션을 종료한다.

□ 개발 중 장애요인과 해결방안

○ 제어 보드와의 통신에서 메시지 큐를 송/수신 공용으로 사용하였을 때 메시지 큐가 종종 불능이 되었다.

프로세스 p가 제어 보드에 명령을 내리기 위해 메시지 큐를 통해 메시지를 전달하고, 제어보드는 p가 보낸 메시지를 받아 처리하고 결과를 메시지 큐를 통해 전달한다. 이런 활동이 반복적으로 행해지면 메시지 큐는 제어보드가 받아야 할 메시지와 보낸 메시지가 뒤섞여 있어서 제어보드 프로세스는 자신이 보낸 메시지를 큐에서 꺼내어 처리하고자 할 것이다. 원활한 명령을 위해 송/수신 메시지 큐를 분리하였다. 또한, 제어보드로부터 답장을 받을 필요가 없는 명령들(목표 속력 설정, 제어 상수 설정 등)은 메시지를 보내고 답장을 기다리지 않게 하고 답장을 받아야 하는 명령들(엔코더 카운터 읽기, 현재 조향각 읽기 등)만 메시지를 기다리게 한다. 프로그램이 메시지를 기다리는 데 할애한 시간을 다른 작업에 사용하여 프로그램의 전체적인 수행능력이 향상되었다.

○ 주차미션에서 미션 발견 즉시 차량을 멈추어도 관성때문에 바로 멈추지 않았다.

주차미션은 도로주행에 비해 느린 속도로 이동하며 정밀한 거리를 이동하여야 한다. 계획한 알고리즘 상으로는 오른쪽 앞의 PSD가 주차공간 끝과 나란할 때 멈추어야 하는데 처리 process 내에서 주차미션을 검사하는 함수가 실행되는 주기가 꽤 길고, 바로 멈춘다고 하여도 빠른 주행속도를 이기지 못하고 수cm이상 앞으로 밀린 후 정지한다. 이를 보완하기 위해 차를 정지한 후 느린 속도로 후진하며 오른쪽 앞의 PSD와 주차공간의 끝을 맞추어 현재 로봇과 주차공간의 상대적인 위치를 정밀하게 파악할 수 있게 했다.

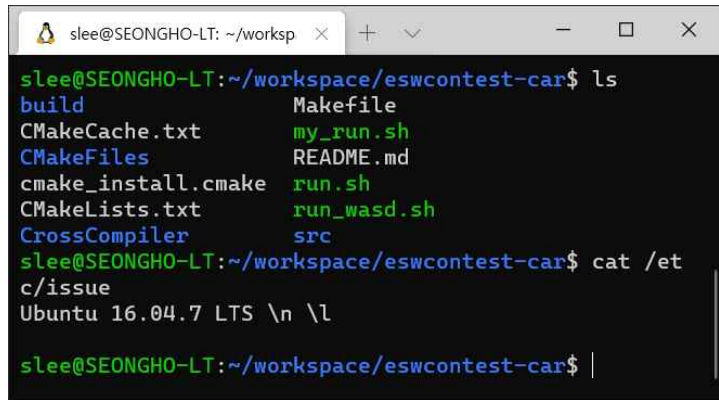
○ 우선정지 장애물이 노이즈로 인해 가끔 인식하지 못하여 우선정지 장애물이 사라졌다고 판단하고 출발해버렸다.

카메라 영상은 주변 조명, 대상의 흔들림 등에 변화하여 인식 결과가 쉽게 바뀔 수 있다. 우선정지 장애물 역시 그러하였다. 장애물의 일시적인 인식장애를 극복하고 확실하게 장애물의 존재여부를 파악하기 위하여 one-shot 필터를 사용하였다. 알고리즘은 간단한데, 루프를 돌면서 우선정지 장애물이 없다면 카운트를 1 증가하고, 장애물이 존재한다면 카운트를 0으로 초기화한다. 카운트에 임계값을 설정하여(예를들어 500) 카운트가 임계값을 넘어서면 루프를 탈출한다. 이 임계값은 실험적으로 도출할 수 있으며 직관적이고 쉽게 변경할 수 있다는 장점이 있다. 장애물이 주변 환경의 영향으로 잠깐 안 보여서 카운트가 10까지 올라갔더라도 루프를 돌다가 장애물을 확인하게 되면 카운트가 0으로 초기화되며 루프를 계속 돈다. 반면 장애물이 완전히 사라지면 카운트가 계속 증가하여 500을 넘기면 루프를 탈출하여 다음 명령을 실행한다. 우선정지 장애물을 영상에서 검출하는 작업은 장애물이 존재하지 않을 때 장애물을 인식할 확률은 매우 희박하지만, 장애물이 실제로 존재함에도 때때로 인식하지 못할 수 있다는 특징이 있어서 이러한 one-shot필터를 사용해주면 효과적이다.

□ 개발의 차별성

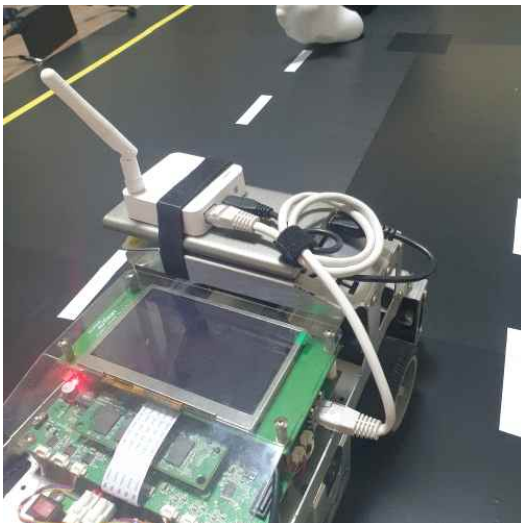
○ WSL

우분투 리눅스 운영체제를 이용하는 방법은 세 가지가 있다. 첫 번째는 컴퓨터에 운영체제를 설치하는 방법인데, 듀얼부팅 시 컴퓨터의 메인보드와 호환이 잘 안 되었다. 두 번째는 현재 사용하고 있는 OS에서 가상머신을 설치하여 그 위에서 우분투를 실행하는 방법인데, 가상머신이 컴퓨터 자원을 너무 많이 소비하여 컴퓨터 동작이 느려져 원활한 작업이 불가능하였다. 마지막으로, 윈도우에서 제공하는 WSL(Windows Subsystem for Linux)를 사용하는 것이다. USB로 minicom접속은 불가능하지만 ssh를 사용하므로 불편함이 없고, 리눅스를 윈도우 응용프로그램처럼 빠르고 간편하게 사용할 수 있다.



```
slee@SEONGHO-LT: ~/worksp
slee@SEONGHO-LT:~/workspace/eswcontest-car$ ls
build                               Makefile
CMakeCache.txt                     my_run.sh
CMakeFiles                         README.md
cmake_install.cmake                run.sh
CMakeLists.txt                     run_wasd.sh
CrossCompiler                      src
slee@SEONGHO-LT:~/workspace/eswcontest-car$ cat /etc/issue
Ubuntu 16.04.7 LTS \n \l
slee@SEONGHO-LT:~/workspace/eswcontest-car$ |
```

○ 무선 개발



기존에 랜 케이블을 이용한 scp 명령어를 통해 실행 파일을 전송하고 minicom으로 셸 명령어를 수행하는 과정이 번거로웠다. 그래서 처음에는 랜 케이블만을 활용해 ssh로 개발을 단순화 하였다. 그러나 랜 케이블을 연결한 채 주행하다보면 케이블의 접촉 불량으로 프로그램이 비정상적으로 종료되어 개발에 큰 어려움이 있었다. 랜 케이블을 대체할 작은 무선AP를 로봇에 부착하여 작업실 내의 공유기에 접속하여 다른 host pc들과 통신할 수 있게 했다. 랜 케이블의 제약에 구애받지 않고 이전보다 안전하게 주행할 수 있게 되었다. 좌측 그림은 보조배터리와 그 위에 무선AP를 올려놓은 모습이다.

○ Build 자동화

개발이 고도화됨에 따라 여러 개의 프로세스를 빌드해주어야 해서 빌드 명령어가 복잡해졌다. 이를 Makefile을 이용하여 간단히 빌드 할 수 있었지만, 새로운 소스코드를 만들 때마다 등록해 주어야 하는 번거로움이 있었다. 이를 개선하기 위해 cmake를 이용하여 이전에 복잡했던 빌드과정을 자동으로 수행하였다. CMakeLists.txt에 설정값과 디렉토리 주소를 입력해 주면 cmake가 알아서 c파일을 찾아서 Makefile을 만들어 주고, 우리는 만들어진 make명령어로 수월하게 빌드할 수 있다. 한편, 빌드된 여러 실행파일들을 일일이 타겟보드에 전송하고 실행하는 과정도 수고스러웠다. run.sh라는 쉘 스크립트를 만들어 빌드와 바이너리 파일 전송과 그 실행까지의 일련의 과정을 한 명령어로 수행할 수 있게 했다.

□ 개발 일정

No	내용	2020年											
		6月			7月			8月			9月		
1	프로젝트 준비 및 방 구하기												
2	CarSDK 분석												
3	ctrlboard 미들웨어 개발												
4	recognize 프로세스 prototype 개발												
4-1	PSD curve-fitting												
4-2	우선 정지 장애물 감지												
4-3	신호등 정보 감지												
4-4	차선의 위치 감지												
4-5	정지선 감지												
4-6	차선을 밟았는지 여부 감지												
4-7	종료 지점 감지												
4-8	내리막길(경사로)에 있는지 여부												
4-9	터널 감지												
4-10	전방 차량 존재 여부												
5	process 프로세스 prototype 개발												
5-1	우선정지 장애물 구간												
5-2	도로주행 및 곡선(S)자 주행 코스												
5-3	수평주차, 수직주차												
5-4	고가도로 구간												
5-5	터널 코스												
5-6	회전 교차로												
5-7	신호등 분기점 코스 및 종료지점												
5-8	돌발(내리막) 구간												
5-9	차로 추월 구간												
6	디버깅												
7	개발 완료 보고서 작성												

□ 팀 업무 분장

No	구분	성명	참여인원의 업무 분장
1	팀장	이성호	- 프로젝트 총괄 및 모듈 통합. - ctrlboard 미들웨어와 recognize 프로세스의 prototype을 개발함. - recognize 프로세스에서 내리막길(경사로)에 있는지 여부를 감지하는 기능 개발. - process 프로세스에서 고가도로 구간 및 돌발(내리막) 구간 미션 수행 로직을 설계함.
2	팀원	권준호	- process 프로세스의 prototype을 개발함. - recognize 프로세스에서 차선의 위치 정보, 전방 차량 존재 유무를 인식하는 기능 개발. - process 프로세스에서 도로주행 및 곡선(S)자 주행 코스, 차로 추월구간 미션 수행 로직을 설계함.
3	팀원	김정현	- recognize 프로세스에서 영상 처리를 통해 신호등과 우선 정지 장애물을 인식하는 기능 개발. - process 프로세스에서 회전 교차로 미션 수행 로직을 설계함.
4	팀원	박재선	- recognize 프로세스에서 curve-fitting을 통해 얻은 거리-PSD값 함수를 적용함. - process 프로세스에서 수평주차 및 수직주차 미션 수행 로직, 신호등 분기점 코스 및 종료 지점 미션 수행 로직을 설계함.
5	팀원	이준서	- recognize 프로세스에서 정지선, 차량의 차선 침범 여부, 종료 지점을 인식하는 기능 개발. - process 프로세스에서 터널 미션 수행 로직 및 출발 신호 미션 수행 로직을 설계함.